

Context-Free Grammars

As we saw last week, regular expressions are well suited to defining simple languages, like search patterns or the basic symbols of a programming language.

As we noted last week, there are some common structures, notably bracketing structures, that cannot be expressed by regular languages.

This week we'll consider context-free grammars, which allow us to express more complex languages, e.g. they are used to define the syntax of most programming languages.

Languages and grammars

Recall that a formal language is a set of strings over an alphabet Σ . Because context-free grammars are typically used at a higher level of structure, we typically use different terminology:

- For the simpler regular expressions, we usually call the elements of Σ *letters*, and the strings of the language *words*.
- For the richer context-free languages, we usually call the elements of Σ *words*, and the strings of the language *sentences*.

Not any old combination of words or symbols is syntactically valid. A grammar is a set of rules for constructing valid "sentences" of the language. As we'll see, a grammar also imposes a structure on these sentences. This is very important for computer processing, as it is the first step in turning human readable text into something a computer can manipulate, e.g. to generate machine executable code.

Approximating natural languages

Try some regular expressions:

Note that being syntactically valid is not (always) the same as being meaningful.

Languages people speak and write follow very complex grammatical rules, but we can approximate them for a simple subset to illustrate the idea of a formal grammar.

Let's think about constructing simple English sentences, from a small vocabulary. We will use the following braced words to name syntactic categories:

{sentence}, {subject}, {predicate}, {noun-phrase},
{article}, {modified-noun}, {adjective}, {noun} and
{verb}.

A formal grammar gives possible expansions of these categories in terms of other categories and concrete words:

{sentence} → {subject} {predicate}
{subject} → {noun-phrase}
{predicate} → {verb} {noun-phrase}
{noun-phrase} → {article} {modified-noun}
{modified-noun} → {adjective} {modified-noun}
{modified-noun} → {noun}
{noun} → apple
{noun} → bear
{noun} → worm
{noun} → dog
{verb} → eats
{verb} → follows
{verb} → eats
{verb} → hugs
{adjective} → wriggly
{adjective} → tasty
{article} → a
{article} → an
{article} → the

The rules of the grammar are equivalent to the following description in natural language:

- A sentence can be a subject followed by a predicate.
- A subject can be a noun phrase.
- A predicate can be a verb followed by a noun phrase.
- A noun phrase can be an article followed by a modified noun.
- A modified noun can be an adjective followed by a modified noun.
- A modified noun can be a noun.
- A noun can be: "apple", "bear", "worm" or "dog".
- A verb can be "eats", "follows", "gets" or "hugs".
- An adjective can be "wriggly" or "tasty".
- An article can be "a", "an" or "the".

We call this a context-free grammar, because the possible expansion of a category like {noun-phrase} does not depend on any words that surround it (i.e. its context).

Formalizing these ideas:

definition

A context-free grammar (CFG) is a collection of four things:

- An alphabet, Σ , of symbols called terminals from which strings in a language will be constructed.
- A separate set of symbols called non-terminals or variables.
- A distinguished non-terminal called the start symbol, usually called S .
- A finite set of productions or rules of the form $A \rightarrow \beta$, where A is a non-terminal and β is a finite sequence of zero or more terminal or non-terminal symbols.

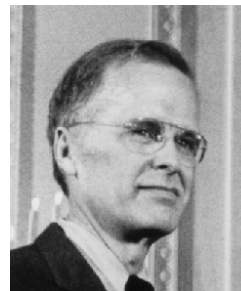
In the example above,

- The terminal symbols are concrete words like apple, bear, worm, dog, etc.
- The non-terminal symbols are the replaceable categories, $\langle \text{sentence} \rangle$, $\langle \text{subject} \rangle$, $\langle \text{predicate} \rangle$, etc.
- The start symbol is $\langle \text{sentence} \rangle$.
- The productions are the rules like
 $\langle \text{sentence} \rangle \rightarrow \langle \text{subject} \rangle$
 $\langle \text{predicate} \rangle$

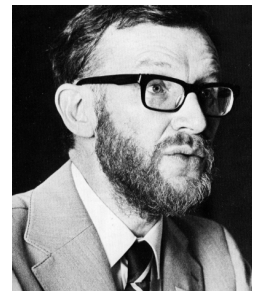
Alternative notation

Sometimes a more compact notation is used, combining rules with the same left-hand side and separating the right-hand sides with vertical bars:

```
 $\langle \text{sentence} \rangle \rightarrow \langle \text{subject} \rangle \langle \text{predicate} \rangle$   
 $\langle \text{subject} \rangle \rightarrow \langle \text{noun-phrase} \rangle$   
 $\langle \text{predicate} \rangle \rightarrow \langle \text{verb} \rangle \langle \text{noun-phrase} \rangle$   
 $\langle \text{noun-phrase} \rangle \rightarrow \langle \text{article} \rangle \langle \text{modified-noun} \rangle$   
 $\langle \text{modified-noun} \rangle \rightarrow \langle \text{adjective} \rangle \langle \text{modified-noun} \rangle \mid$   
 $\langle \text{noun} \rangle$   
 $\langle \text{noun} \rangle \rightarrow \text{apple} \mid \text{bear} \mid \text{worm} \mid \text{dog}$   
 $\langle \text{verb} \rangle \rightarrow \text{eats} \mid \text{follows} \mid \text{hugs}$   
 $\langle \text{adjective} \rangle \rightarrow \text{wriggly} \mid \text{tasty}$   
 $\langle \text{article} \rangle \rightarrow \text{a} \mid \text{an} \mid \text{the}$ 
```



John Backus



Peter Naur

This notation is known as Backus-Naur form (BNF), after the two computer scientists who used it to describe the syntax of the pioneering programming language Algol 60.

Backus and Naur wrote '::<=' instead of ' $\rightarrow$ '. Some books use '=' or '!'.

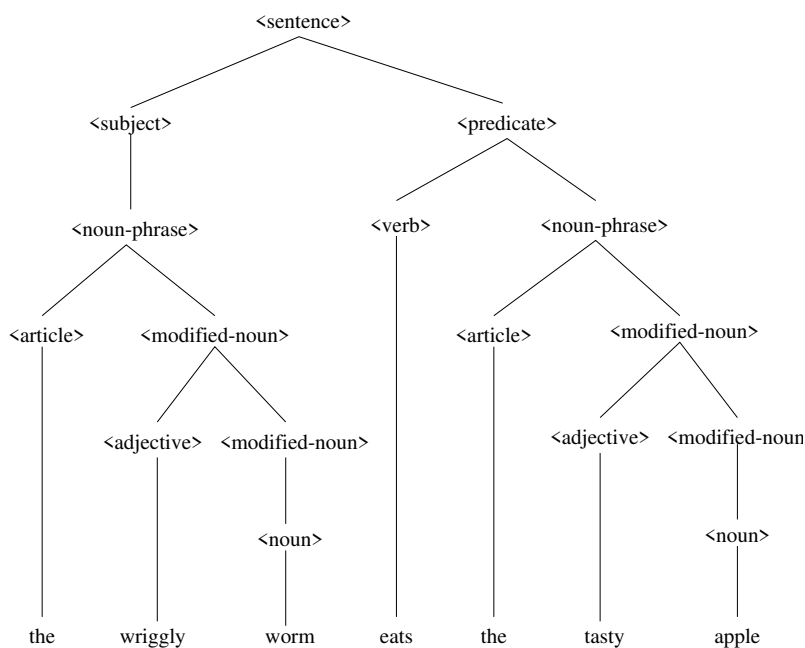
Deriving a sentence

The language described by the above grammar includes the

sentence:

The wriggly worm eats the tasty apple

We can show this with a derivation tree, starting with the start symbol $\langle \text{sentence} \rangle$ and expanding a non-terminal symbol (here marked with angle brackets), such as $\langle \text{noun} \rangle$, using a rule from the grammar until the only terminal symbols (concrete words) remain:



Computer scientists usually draw trees upside-down, with the root at the top and the leaves at the bottom.

- Each internal node of the tree is labelled with a non-terminal. (A non-terminal can occur more than once in the tree.)
- The root of the tree is labelled with the start symbol.
- The leaves of the tree (at the bottom) are labelled with non-terminals.
- The children of each non-terminal correspond to the right-hand side of a production of that non-terminal.

$\langle \text{sentence} \rangle \rightarrow \langle \text{subject} \rangle$

$\langle \text{predicate} \rangle$

$\langle \text{subject} \rangle \rightarrow \langle \text{noun-phrase} \rangle$

$\langle \text{predicate} \rangle \rightarrow \langle \text{verb} \rangle \langle \text{noun-phrase} \rangle$

$\langle \text{noun-phrase} \rangle \rightarrow \langle \text{article} \rangle$

$\langle \text{modified-noun} \rangle$

$\langle \text{modified-noun} \rangle \rightarrow \langle \text{adjective} \rangle$

$\langle \text{modified-noun} \rangle \mid \langle \text{noun} \rangle$

$\langle \text{noun} \rangle \rightarrow \text{apple} \mid \text{bear} \mid \text{worm} \mid \text{dog}$

$\langle \text{verb} \rangle \rightarrow \text{eats} \mid \text{follows} \mid \text{hugs}$

$\langle \text{adjective} \rangle \rightarrow \text{wriggly} \mid \text{tasty}$

$\langle \text{article} \rangle \rightarrow \text{a} \mid \text{an} \mid \text{the}$

Formally:

Derivation is sometimes presented as

A derivation tree for a context-free grammar is a tree such that

- Each node is labelled with a terminal or non-terminal symbol.
- The root of the tree is labelled with the start symbol.
- A node labelled with a non-terminal symbol A has children labelled left-to-right with the symbols in β , where $A \rightarrow \beta$ is a production of the grammar.
- Nodes labelled with terminal symbols are leaves (i.e. have not children).

definition

A string w can be derived from the grammar if there is a derivation tree for the grammar whose leaves, read left-to-right, comprise the string w .

definition

The language of a context-free grammar is the set of strings of terminals that can be derived from the grammar.

beginning with the start symbol and repeatedly expanding non-terminals until only terminals are left, e.g.:

$\langle \text{sentence} \rangle$
 $\Rightarrow \langle \text{subject} \rangle \langle \text{predicate} \rangle$
 $\Rightarrow \langle \text{noun-phrase} \rangle \langle \text{predicate} \rangle$
 $\Rightarrow \langle \text{article} \rangle \langle \text{modified-noun} \rangle$
 $\langle \text{predicate} \rangle$
 $\Rightarrow \dots$
 $\Rightarrow$ The wriggly worm eats the
tasty $\langle \text{noun} \rangle$
 $\Rightarrow$ The wriggly worm eats the
tasty apple

Because the grammar is context-free, we could expand the braced words in any order, and still get the same result.

Programming languages

Context-free grammars are commonly used to describe the syntax of programming languages. For another example, here is part of a (simplified) grammar for Java:

```
 $\langle \text{statement} \rangle \rightarrow ;$   
 $\langle \text{statement} \rangle \rightarrow \langle \text{expression} \rangle ;$   
 $\langle \text{statement} \rangle \rightarrow \{ \langle \text{statement-list} \rangle \}$   
 $\langle \text{statement} \rangle \rightarrow \text{while} ( \langle \text{expression} \rangle ) \langle \text{statement} \rangle$   
 $\langle \text{statement} \rangle \rightarrow \text{do} \langle \text{statement} \rangle \text{while} ( \langle \text{expression} \rangle ) ;$   
...  
 $\langle \text{statement-list} \rangle \rightarrow \epsilon$   
 $\langle \text{statement-list} \rangle \rightarrow \langle \text{statement} \rangle \langle \text{statement-list} \rangle$ 
```

Algorithms

definition

A recognizer for a grammar is an algorithm that tells us

In the Language Processors module

whether or not an input string belongs to the language, i.e. has a derivation from that grammar.

(if you take it), you'll be using a parser generator, a program that takes an appropriate context-free grammar and writes an efficient parser for the grammar.

Parsers can also be written by hand using a technique called recursive descent, as described in the optional extra material.

More often, one wants to go in the opposite direction, starting with a string and working out how it can be derived from the start symbol. This process is called parsing, and in that context the tree would be called a parse tree.

Because such trees impose a structure on the string, they are often the starting point for subsequent processing.

definition

A parser for a context-free grammar is an algorithm that takes as input a string and either

- returns a parse tree for the string, or
- announces that no parse tree exists for that string.

General parsing algorithms parse a string of length n in $O(n^3)$ time. In practice, one uses special kinds of context-free grammar that have $O(n)$ time parsers, but we won't describe those here.

Example grammars

Anything

In this example, we take the alphabet $\Sigma = \{a, b\}$.

For the derivation to terminate, the grammar must include production rules whose right-hand sides contain terminals only. In this example, the null string is used to terminate the derivation.

Try the production rules:

S

→

aS | bS | ε

All derivations

Derivation of

aba

This grammar can produce many strings, including aba. Try other strings.

Try changing the grammar, e.g. replacing aS with Sa . Does the changed grammar produce the same language?

This grammar can produce any string composed of a's and b's, i.e. it describes the same language as the regular expression $(a|b)^*$.

Try another grammar for the same language:

S	→	XS ϵ	All derivations
X	→	a b	
Derivation of		aba	

Again, try changing XS to SX.

Bracketing

For this example, we assume the alphabet $\Sigma = \{a, b, c\}$.

Try the production rules:

S	→	aSb SS c	All derivations
Derivation of		acbaacbb	

If we think of "a" as "(" and "b" as ")", this is the language of properly nested parentheses.

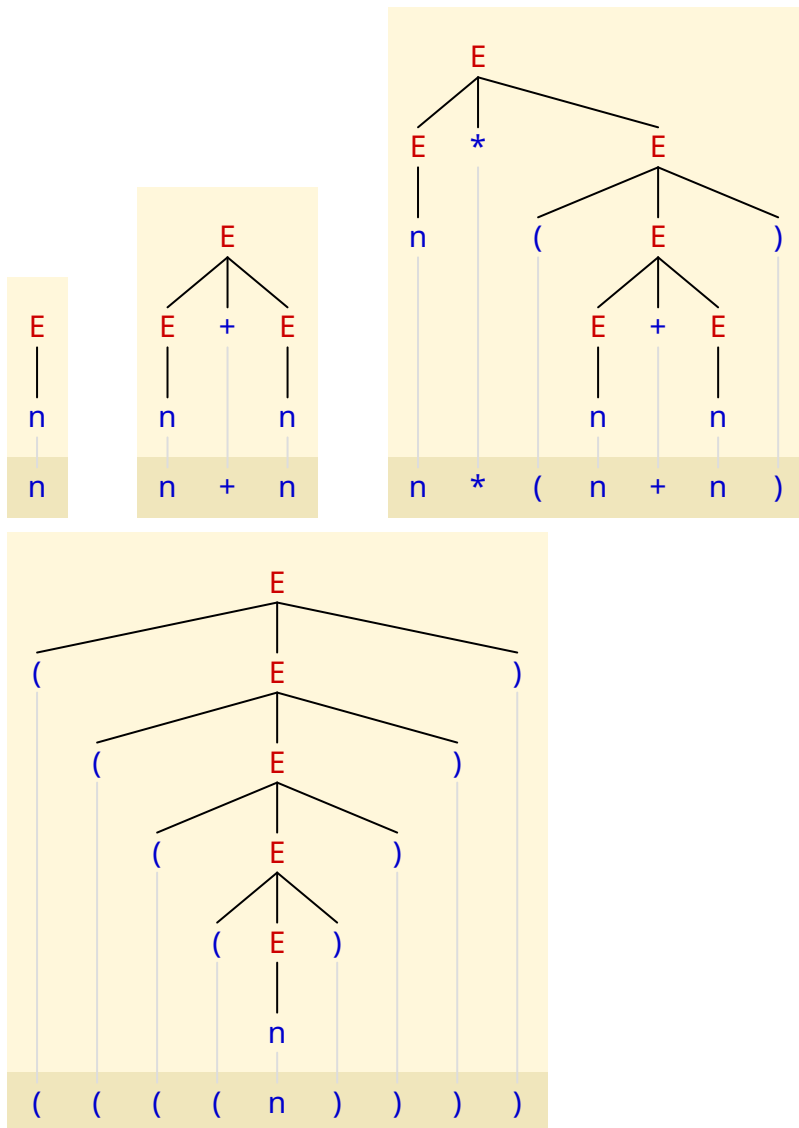
Arithmetic expressions

To define a grammar for arithmetic expressions, we use an alphabet $\Sigma = \{+, -, *, /, (,), n\}$, where n stands for numbers.

Then the following grammar defines expressions:

$$E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid (E) \mid n$$

Here are some derivations of strings from this grammar:



Regular languages and context-free languages

Any regular language can be described using a context-free grammar. Indeed we can express Kleene star α^* (zero or more α 's) as a production in two ways, either

$$X \rightarrow \varepsilon \mid \alpha X$$

or, equivalently

$$X \rightarrow \varepsilon \mid X \alpha$$

Try regular expressions:

Try the grammar:

For example, let's start with the regular expression $(a|b)((a|b)(a|b))^*$. We introduce non-terminals to stand for the various parts of the expression, and build up until we have a grammar for the whole thing:

- Let X stand for $(a|b)$. We can define it with a production

$$X \rightarrow a \mid b$$

- Let Y stand for $((a|b)(a|b))^*$, which we can write more briefly using the previous definition as $(XX)^*$. We can express that as either

$$Y \rightarrow \varepsilon \mid XXY$$

or, equivalently

$$Y \rightarrow \varepsilon \mid YXX$$

- Now if S stands for the whole expression, we can define it with

$$S \rightarrow XY$$

S	→
XY	
Y	→
ε XXY	
X	→
a b	

Right linear grammars

In fact there's a mechanical translation from finite state machines to a restricted form of context-free grammar called a right linear grammar or a right regular grammar, in which all productions have one of the two forms $A \rightarrow aB$ or $A \rightarrow \varepsilon$.

For example, consider the regular expression $(a|b)a((a|b)a)^*$, which describes the language of non-empty strings of even length, with every second letter being a .

Try this regular expression:

This regular language can be expressed as the finite state machine at right.

But not all context-free languages are regular

Let's derive a context-free grammar from this machine. For each state, we will introduce a corresponding non-terminal

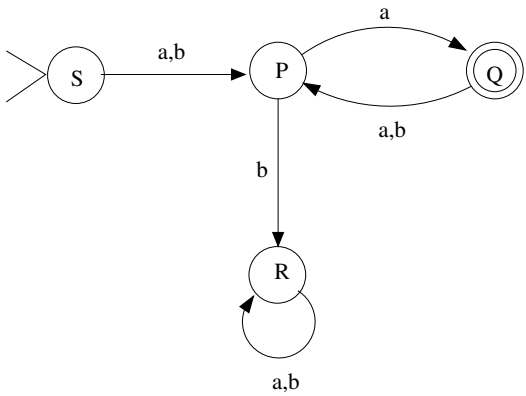
standing for the set of strings that take us from that state to an accept state. We can ignore state R, since it never leads to an accept state. We create a grammar with one non-terminal for each of the other states, turning the transitions into productions:

$$\begin{aligned}
 S &\rightarrow a P \mid b P \\
 P &\rightarrow a Q \\
 Q &\rightarrow \epsilon \mid a P \mid b P
 \end{aligned}$$

Q has the empty string ϵ as an alternative because Q is an accept state. Because S is the start state of the machine, S is the start symbol of the grammar.

This isn't the only context-free grammar for this language. We can derive a simpler one directly from the regular expression, defining $X = (a|b)a$ and $S = X^+$:

$$\begin{aligned}
 S &\rightarrow X \mid X S \\
 X &\rightarrow aa \mid ba
 \end{aligned}$$



Try the grammar:

S

→

a P | b P

P

→

a Q

Q

→

ε | a P | b P

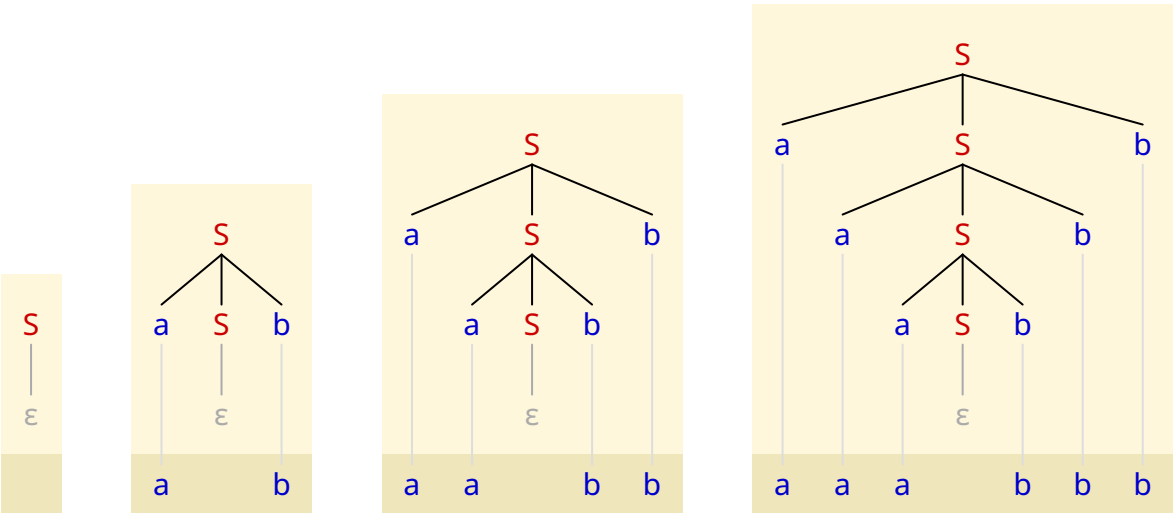
All derivations

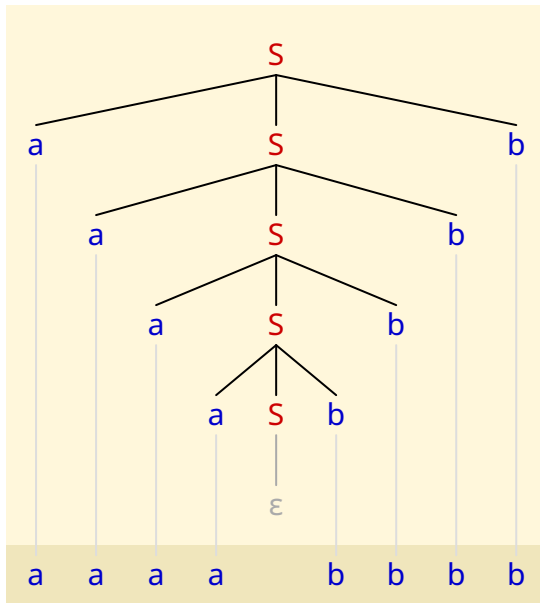
Derivation of

For example, consider the context-free grammar:

$$S \rightarrow aSb \mid \epsilon$$

This grammar allows the following derivations:





So it generates the language $\{\epsilon, ab, aabb, aaabbb, aaaabbbb, \dots\} = \{a^n b^n\}$. This is not a regular language, because it involves counting the letters in two components to ensure they are balanced.

That is, context-free grammars can describe more languages than regular expressions.

The Chomsky hierarchy revisited

The different types of languages in the Chomsky hierarchy are described by grammars that allow different types of productions:

Type	Grammars	Productions	Recognizers
0	general	$\alpha \rightarrow \beta$	Turing machines
1	context-sensitive	$\alpha X \beta \rightarrow \alpha \gamma \beta$	linear-bounded Turing machines
2	context-free	$X \rightarrow \gamma$	Pushdown machines
3	regular	$X \rightarrow a B$	Finite state

An example of a language that is not context-free (but is context-sensitive):

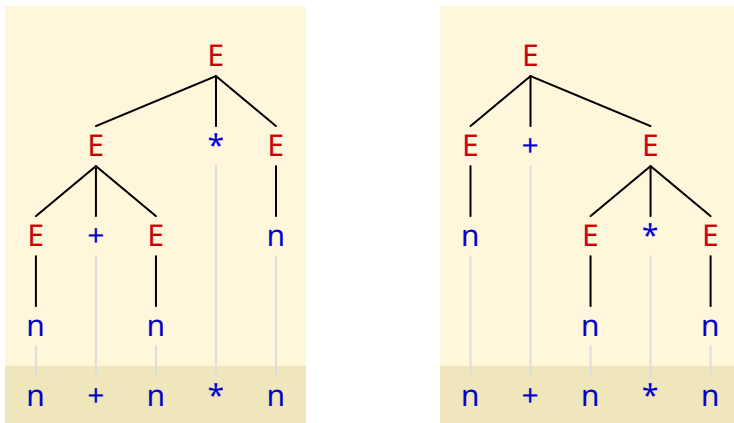
$$\{\epsilon, abc, aabbcc, aaabbbccc, aaaabbbbcccc, \dots\} = \{a^n b^n c^n\}.$$

Ambiguity

Recall the grammar above for expressions:

$$E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid (E) \mid n$$

There are two parse trees for the string $n + n * n$:



That is, there are two different ways we can parse this string using this grammar (and they lead to different meanings).

definition

A context-free grammar is ambiguous if some string has more than one parse tree.

We usually want to avoid ambiguous grammars. Fortunately we can usually construct an equivalent unambiguous grammar for the same language.

The expression language has several sources of ambiguity:

- combinations like $E + E * E$ or $E * E + E$

- combinations like $E + E - E$ or $E - E + E$
- combinations like $E * E / E$ or $E / E * E$

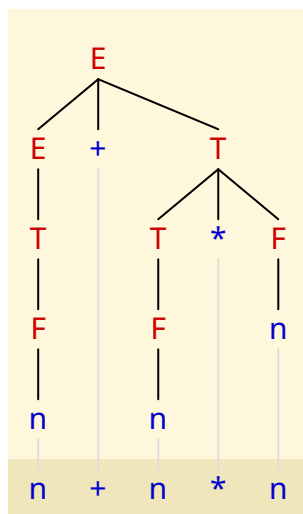
We need to tighten the grammar so that it chooses the parse that one would expect using standard rules:

- $*$ and $/$ bind more tightly than $+$ and $-$
- a combination of $+$ and $-$ associates to the left
- a combination of $*$ and $/$ associates to the left

The following grammar incorporates these rules, making it unambiguous, but still describes the same language:

$$\begin{aligned} E &\rightarrow E + T \mid E - T \mid T \\ T &\rightarrow T * F \mid T / F \mid F \\ F &\rightarrow (E) \mid n \end{aligned}$$

With this grammar, the above string has only one parse tree:



However, there exist context-free languages that have no unambiguous grammar. Such languages are said to be inherently ambiguous.

Decision problems

There exist algorithms to test whether the language represented by a context-free grammar is empty, finite or infinite. However, many other properties of context-free grammars are undecidable. That is, there is no general

Try the revised grammar:

E	→	
E + T E - T T		
T	→	
T * F T / F F		
F	→	
(E) n		

All derivations

Derivation of

The non-terminals stand for expressions, terms and factors, respectively. Try other strings.

procedure for any of these problems:

Ambiguity

Is the grammar ambiguous?

Universality

Is the language equal to Σ^* ?

Equality

Do two grammars describe the same language?

Inclusion

Is the language described by one grammar a subset of the language described by another?

Overlap

Is there a string derivable by both of two grammars?
